

# 《Web后端开发技术》期末大作业指导书

## (2026)

### 一、作业目标与核心指导思想

本次大作业旨在全面考察同学们对现代微服务架构、主流中间件（Redis/MQ）、现代Web开发技术以及AI大语言模型（LLM）集成的综合运用能力。

 **核心开发策略：“重架构设计，轻底层搬砖”**

- 拥抱 AI 辅助编程：**本次作业允许且鼓励使用Qoder、Trae、Codex、Claude Code 等 AI Coding 工具辅助编写代码。对于系统中的“基础增删改查（CRUD）”代码，强烈建议利用 AI 快速生成，节约时间。
- 死磕核心架构（给高分关键）：**本课程的核心是高并发处理、消息中间件与系统架构。系统的成败取决于当基础CRUD遇到“秒杀”、“海量数据交互”、“耗时AI调用”时，你的架构设计是否合理。**如果你只写了一堆完美的CRUD直接操作数据库，没有使用网关、Redis、MQ和实时通信，将会严重影响分数。**
- 无需前端界面：**受限于时间和重点考查方向，本项目不需要编写前端代码。所有流程验证可通过 API 调试工具（Postman/Apifox等）完成，也可以使用AI生成简单的前端测试页面。

### 二、题目选择（三选一）

请从以下三个业务场景中任选其一完成系统的原型开发与架构设计。每个题目均包含“基础支撑功能（打底数据）”和“核心架构功能（重点考核）”。

#### 题目一：AI驱动的“高并发秒杀与智能电商”后台系统

**场景描述：**模拟一个包含日常商品浏览和高并发秒杀活动的电商后台，结合AI为用户提供智能导购。系统需拆分如：用户服务、商品/库存服务、订单服务、秒杀服务等。

- 第一部分：基础支撑功能（建议用AI快速完成）**
  - 用户权限：**提供注册/登录接口，区分普通客户和管理员两类角色权限，登录成功颁发并校验 JWT Token，支持注销登录。
  - 商品与库存管理：**商品的添加、下架、列表分页查询；设置商品的初始库存并落盘到关系型数据库。
  - 日常订单管理：**查询个人的订单列表、查看订单状态（待支付、已创建等）。
- 第二部分：核心架构功能（重点考核技术点）**

- a. **统一入口控制**：所有客户端请求必须经过API服务网关（Gateway）路由到具体服务。
  - b. **高并发防超卖**：针对100件库存、10000个并发的秒杀场景，**禁止**直接Update数据库。必须使用Redis缓存预热库存，并完成原子扣减。
  - c. **异步订单削峰**：Redis扣减成功后，立即返回“排队中”。通过消息队列将请求异步发送给订单服务，完成最终订单落盘。
  - d. **抢购结果实时推送**：订单真正在数据库创建成功后，后端通过WebSocket或SSE主动向客户端推送“订单创建成功”的消息。
  - e. **AI智能导购**：提供商品咨询API，后端获取商品描述上下文，结合用户提问调用**大语言模型（LLM）API**生成购买建议返回。
- 

## 题目二：企业级“智能工单与实时协作”管理后台

**场景描述**：设计一个面向企业内部的工单流转系统，AI作为“虚拟员工”参与工单的初步分类与处理，紧急故障需实时预警。

- **第一部分：基础支撑功能（建议用AI快速完成）**
    - a. **角色管理**：提供注册/登录接口，区分普通客户和管理员两类角色权限，登录成功颁发并校验JWT Token，支持注销登录。
    - b. **工单基础流转**：客户侧提供提交工单（标题、描述等）、查询个人工单状态的API；客服侧提供拉取所有工单、更新状态（处理中、已解决）、追加回复记录的API。
    - c. **FAQ管理**：管理员增删改“常见问题及标准话术”列表。
  - **第二部分：核心架构功能（重点考核技术点）**
    - a. **统一入口控制**：所有客户端请求必须经过API服务网关（Gateway）路由到具体服务。
    - b. **服务注册与发现**：拆分网关、工单、AI等微服务，使用**服务注册中心**进行管理，服务间可互相调用，杜绝硬编码IP端口。
    - c. **常见问题智能问答**：提供AI问答服务，客户可以通过AI按照FAQ里面的内容进行智能回复。
    - d. **工单AI异步预处理**：客户提交几百字故障描述后，API立即返回成功。随后通过**消息中间件（MQ）**触发后台耗时的AI处理，调用**LLM API**提取故障分类和紧急程度，并写回该工单数据库。
    - d. **紧急工单实时告警**：一旦AI将某工单判定为“高紧急程度”，后台需通过**WebSocket或SSE**第一时间将警告推送给在线客服。
- 

## 题目三：基于AI的“智能知识库与内容发布”平台

**场景描述**：类似于具有AI能力的知乎后台。包含文章发布、互动点赞、动态热榜，并提供沉浸式的AI写作辅助。

- **第一部分：基础支撑功能（建议用AI快速完成）**
    - a. **用户权限：** 提供注册/登录接口，区分普通客户和管理员两类角色权限，登录成功颁发并校验JWT Token，支持注销登录。
    - b. **文章内容管理：** 文章的创建（草稿）、发布、修改、逻辑删除；分页获取最新文章列表、获取文章详情API。
    - c. **互动基础功能：** 简单的评论入库API，以及单次点赞更新数据库API。
  - **第二部分：核心架构功能（重点考核技术点）**
    - a. **动态网关限流：** 为防恶意爬虫，在**网关层需支持**配置限流策略（如限制某IP对文章详情API的访问频率）。限流参数需由**服务配置中心**动态下发。
    - b. **文章实时热搜榜：** 用户阅读或点赞文章，该文章热度上升。可使用Redis的Sorted Set（ZSET）数据结构维护并提供“全站热度Top 10文章榜单”API。
    - c. **长文异步广播处理：** 用户发布长文后，投递消息至**消息队列**。采用发布/订阅（或Fanout）模式，让多个下游服务**同时、独立**处理同一篇文章（如：服务A调AI提取标签，服务B调用AI进行合规检测等）。
    - d. **AI流式创作辅助：** 提供“AI续写”API，须采用**流式响应（Streaming API）**，结合现代异步Web框架或SSE（Server-Sent Events）技术，将AI生成的文字逐字/逐句推给客户端。
- 

### 三、验收要求

本次作业由于无需部署到真实的公有云集群，故验收分为“本地原型系统验收”和“生产环境架构文档设计”两部分。

#### 1. 系统开发实现（占总分50%）

- **代码提交：** 提交完整的源代码工程，必须体现**微服务架构（至少3个独立服务）**。
- **工具证明：** 必须使用真实的关系型数据库（MySQL/PostgreSQL）、Redis、MQ和网关等中间件。
- **API文档：** 提供自动生成的API文档（Swagger/OpenAPI）。

#### 2. 测试配置与演示视频（占总分10%）

- 提交一份包含核心业务链路的 **Postman Collection**（或 APIFox 导出文件），方便老师/助教一键导入，按顺序调用验证流程。
- 录制一段 **3-5分钟的屏幕录制视频**：
  - 展示各个微服务和中间件的启动状态。
  - 演示主要业务链路的流转（例如：通过Postman触发高并发秒杀请求 -> 在控制台展示MQ日志 -> 客户端收到成功推送 -> 数据库确认数据变更）。

### 3. 期末大作业报告文档中关于系统架构设计与生产环境演进的说明（核心考察点，占总分40%）

代码可以在本地跑通只是第一步。你必须在文档中向老师证明你掌握了处理“高并发、高可用”的理论方案。文档必须包含以下章节：

1. **本地原型架构图与数据流：** 画图描述你本地代码的服务组件结构，以及核心流程的时序图。
2. **数据库ER设计图：** 展示基础数据的关联关系。
3. **技术选型依据分析：** 请不要罗列网上的通稿，结合你的业务场景解释。例如：“我为什么用Redis的ZSET做热榜”、“我的系统里如果MQ挂了会导致什么后果，我是如何设计的”。
4. **【重点】正式生产环境演进方案：** 假设你的系统拥有百万日活，受限于你的本地电脑你无法模拟，但在文档中请说明在正式环境下你会如何改造部署：
  - **高可用（HA）设计：** 如何解决单点故障？（讨论微服务多实例部署、网关集群、数据库主从复制、Redis哨兵/集群模式部署）。
  - **高并发应对扩容策略：** 流量突增时，如何在生产环境中动态扩容、MQ集群等保障系统稳定。
  - **大模型调用的生产级优化：** 第三方LLM API通常有限流限制且响应慢。在并发量极大的情况下，你打算如何在生产环境中优化AI调用链路（如请求排队机制、相似结果缓存等）。

## 四、最终提交物清单

1. 作业报告（报告的核心内容要求见第6部分）
2. 源代码
3. 3-5分钟演示视频

## 五、其它要求

- 分组完成，建议2-3人一组，需在提交报告时体现分工

## 六、作业报告核心章节内容要求

### 一、封面与摘要

- 题目选择与系统名称（题目一/二/三）
- 组员与分工、联系方式
- 代码仓库地址
- 摘要（500字左右）：业务目标、运行环境（语言/框架版本、MySQL/Redis/MQ/网关/注册中心版本）、核心技术点清单（网关、Redis、MQ、实时推送、LLM集成等）和成果亮点（例如：秒杀0超卖、热榜QPS、AI流式推送等）

## 二、业务背景与范围界定

所选题目与业务场景简述

核心用户角色与用例列表（普通用户、管理员、客服等）

MVP 范围：本地原型实现了哪些基础支撑功能；核心架构功能清单

非目标范围（明确不做的功能）

## 三、本地原型系统架构与数据流

组件/微服务清单与职责

中间件清单

架构与部署图

本地部署拓扑：各服务、端口、依赖的中间件关系

核心流程时序图（按题目选择至少包含以下）

题目一：秒杀下单链路（网关→秒杀服务Redis原子扣减→MQ→订单服务落库→WebSocket/SSE推送）

题目二：工单提交→MQ异步AI预处理→写回工单→高紧急实时告警推送；FAQ智能问答

题目三：网关限流→文章阅读/点赞→Redis ZSET热榜更新→Top10查询；文章发布→MQ Fanout→多下游AI处理；AI续写流式推送

安全与网关策略

## 四、数据库设计与持久化策略

ER 图（必须）

用户、角色/权限、商品与库存/工单/文章、订单或处理记录、消息/幂等表、FAQ 等

关键表结构与约束

主键与唯一索引设计

典型SQL与索引说明（为什么加这个索引）

事务与一致性

## 五、缓存、消息与并发控制策略

根据实现方案选择描述，例如

Redis 使用

用途与数据结构：库存预热与Lua原子扣减（题目一）、ZSET热榜（题目三）、缓存维度与TTL

MQ 使用

选型与Topic/队列规划、消息体结构、消费模式（单播/发布订阅）

并发保护

限流（令牌桶/漏桶，网关与服务侧）、熔断隔离、超时与重试回退

## 六、AI/LLM 集成设计

模型与供应商：模型名称、接口协议、速率限制等基本情况

调用方式

## 七、实时通信设计

## 八、技术选型依据与权衡

以上实现方案中各种技术选型的依据与权衡的思考。

## 九、测试

Postman/Apifox 流程说明：按顺序的调用脚本（含环境变量）

功能验证截图/日志

关键链路的请求/响应

性能测试：使用的压测压测工具，并发设置、QPS、成功率、P95/99 延迟

## 十、生产环境演进方案

## 十一、AI 辅助设计开发总结：

核心场景的“人机协作迭代”记录；个人反思